Basic (typed function + custom error) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Where this exercise is saved: `practice/custom_errors_practice.py`, under its `# Basic` section — the same file also holds the Failure handling exercise, under a `# Failure handling` section. Run it with `cd practice && python custom_errors_practice.py`.

Used later by: the Real-world wiring exercise and the Build Task. Neither imports this file — they re-write the same one-line custom-error pattern as `MissingConfigError` in `practice/config_logging_wiring/exceptions.py` and `practice/build_task/exceptions.py`.

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The exercise is really about two Python features working together: a custom exception class, and full type hints on a function.

A custom exception is a class that inherits from `Exception`:

```
# practice/custom_errors_practice.py
class InvalidAgeError(Exception):
    pass
```

`pass` means the class body is empty — you don't need any extra code inside it for a basic custom exception. The class *existing*, with this specific name, is what makes it useful: it lets calling code catch this *one* problem specifically, instead of catching every possible exception with a broad `except Exception`.

The function's type hints should describe its contract precisely:

```
# practice/custom_errors_practice.py
def set_age(age: int) -> None:
```

This tells any reader — without looking at the function body — that it takes an `int` and returns nothing (`None`).

Look specifically at:

- **Exception subclassing:** you're inheriting all of `Exception` 's behavior (the message, the traceback machinery) for free, and only adding a distinct *name* the code can catch.
- **The `raise` statement:** `raise InvalidAgeError(f"age cannot be negative: {age}")` — an f-string here lets you embed the actual bad value in the message.
- **Catching it specifically:** `except InvalidAgeError as e:` — binding the caught exception to `e` lets you access `str(e)` (its message).

Hint 2 — The plan, and almost the whole thing

```
define:
    class InvalidAgeError(Exception): pass

define:
    def set_age(age: int) -> None:
        if age < 0:
            raise InvalidAgeError(f"age cannot be negative: {age}")
        # implicitly returns None if age is valid — no else branch needed

test:
    set_age(25)                          # runs to completion, no output
    try:
        set_age(-5)
    except InvalidAgeError as e:
        print(f"Caught: {e}")
```

Notice there's no `else` branch needed — if the `if` condition is false, the function just falls through and returns `None` naturally.

```
# practice/custom_errors_practice.py
class InvalidAgeError(Exception):
    pass

def set_age(age: int) -> None:
    if age < 0:
        raise InvalidAgeError(f"age cannot be negative: {age}")
```

What's missing: the two test calls. Write them yourself:

```
# practice/custom_errors_practice.py
set_age(25)  # should run with no output

try:
    set_age(-5)
except InvalidAgeError as e:
    ...  # what goes here?
```

Finish it yourself, then compare against the Solution.

Full solution: Show me the solution